
Software Requirements Specification

Music Streaming & Social Network Application
(*Sepatify*)

Course: Mobile Devices Programming
Instructor: Dr. Masoumeh Tarami Rad
Faculty: Computer Engineering
University: Amirkabir University of Technology (AUT)
Semester: Spring 2026
Document ver: 1.0
Date: June 2026

Name	Role	Student ID
Mahan Zavvari	Team Lead / UI & Social	—
Shayan Shahmohammadi	Media Engine & Playback	—
Amirabbas Entezari	Backend & Data Layer	—

Revision History

Version	Date	Author(s)	Description
1.0	June 2026	Mahan Esmaeili	Initial release — full requirements baseline

Contents

Revision History	1
1 Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Definitions, Acronyms, and Abbreviations	4
1.4 References	4
1.5 Document Overview	4
2 Overall Description	5
2.1 Product Perspective	5
2.2 Product Functions — Summary	5
2.3 User Classes and Characteristics	6
2.4 Operating Environment	6
2.5 Design and Implementation Constraints	6
2.6 Assumptions and Dependencies	6
3 Functional Requirements	8
3.1 Authentication & User Management	8
3.2 Home Tab	9
3.3 Search Tab	9
3.4 Downloads Tab	9
3.5 Playlists Tab	10
3.6 Profile Tab	10
3.7 Advanced Media Playback	11
3.8 Now Playing Screen	11
3.9 Business Logic — Premium & Downloads	12
3.10 Social Network & Real-Time Chat	12
3.11 Local Data Management	12
4 Non-Functional Requirements	13

4.1	Usability	13
4.2	Design System	13
4.3	Performance	14
4.4	Architecture & Maintainability	14
4.5	Reliability & Offline Support	14
5	Use Cases	15
5.1	Actors	15
5.2	Use Case Catalogue	15
5.2.1	Authentication Use Cases	15
5.2.2	Content & Playback Use Cases	16
5.2.3	Download Use Cases	16
5.2.4	Social Use Cases	17
5.2.5	Settings Use Cases	17
5.3	Detailed Use Case: UC-12 — Play a Song	18
5.4	Detailed Use Case: UC-41 — Send Message	19
6	Architecture & Technical Design	20
6.1	Layered Architecture	20
6.2	Technology Stack	20
6.3	Key Integration Contracts	21
7	Work Breakdown Structure & Team Assignments	22
7.1	Team Overview	22
7.2	Collaborator A — Backend & Data Layer	22
7.3	Collaborator B — Media Engine & Now Playing	22
7.4	Mahan Esmaeili — UI, Social & State Management	23
8	Development Timeline	24
8.1	Critical Path	24
A	Expected Deliverables	26
B	Song Metadata Schema	27
C	Three Team-Wide Decisions — Week 1 Checklist	28

Chapter 1

Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for the *Music Streaming & Social Network Application*, the capstone project for the **Mobile Devices Programming** course at Amirkabir University of Technology, Spring 2026.

The SRS defines:

- All functional and non-functional requirements for the Android application;
- Architectural and design constraints imposed by the course specification;
- Use cases covering all user interactions;
- A work breakdown structure and task assignments for the development team.

The primary audience is the three-person development team and the course instructor.

1.2 Scope

The system is a full-featured Android application that combines:

1. A **music streaming platform** (analogous to Spotify/Melodify) supporting background playback, offline caching, and advanced media controls;
2. A **social network** layer enabling users to follow each other, share songs, and communicate via real-time direct messages.

The application targets **Android** exclusively and is built with **Jetpack Compose**, **Clean Architecture**, and a developer-owned backend (Firebase, Supabase, Ktor, or Node.js).

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
AUT	Amirkabir University of Technology
UC	Use Case
UDF	Unidirectional Data Flow
MVVM	Model–View–ViewModel
MVI	Model–View–Intent
DI	Dependency Injection
DM	Direct Message
RTL / LTR	Right-to-Left / Left-to-Right layout direction
FR	Functional Requirement
NFR	Non-Functional Requirement
API	Application Programming Interface
WBS	Work Breakdown Structure

1.4 References

- IEEE Std 830-1998, *Recommended Practice for SRS*
- IEEE Std 29148-2018, *Systems and Software Requirements Processes*
- Course project specification — Dr. Masoumeh Tarami Rad, Spring 2026
- Android Jetpack documentation: <https://developer.android.com/jetpack>
- Material Design 3: <https://m3.material.io>

1.5 Document Overview

Chapter 2 — Overall system description and stakeholder context

Chapter 3 — Functional requirements by feature module

Chapter 4 — Non-functional requirements

Chapter 5 — Use case specifications

Chapter 6 — Technical architecture and design constraints

Chapter 7 — Work breakdown and team assignments

Chapter 8 — Development timeline

Chapter 2

Overall Description

2.1 Product Perspective

The application is a self-contained Android system. It communicates with a developer-owned backend service for authentication, song metadata, streaming URLs, and social graph data. No third-party music-streaming API (e.g. Spotify API) is used; all content is self-hosted.

System Boundaries: Android app ↔ Custom backend (REST + WebSocket) ↔ File storage (audio & images). Local persistence is provided by Room (structured data) and DataStore (preferences).

2.2 Product Functions — Summary

The following top-level feature groups are identified:

1. **Authentication & User Management** — register, login, profile, premium
2. **Content Discovery** — home feed, search, playlists
3. **Media Playback** — streaming, background service, controls
4. **Downloads** — premium-gated offline storage via WorkManager
5. **Social Network** — follow/unfollow, public playlists
6. **Real-Time Chat** — WebSocket DMs, song sharing, message status
7. **Settings & Personalisation** — theme, language, account

2.3 User Classes and Characteristics

User Class	Description	Distinguishing Capabilities
Guest	Unauthenticated visitor	Browse landing screen only; must register to use the app
Regular User	Authenticated free-tier user	Stream all songs; search; follow users; chat; cannot download
Premium User	Authenticated paid-tier user (simulated)	All regular capabilities plus offline download
System	Background services and workers	MediaSession playback; WorkManager downloads; WebSocket events

2.4 Operating Environment

- **Platform:** Android (minimum SDK to be decided by team; recommend API 26+)
- **Language:** Kotlin
- **UI Toolkit:** Jetpack Compose with Material 3
- **Backend:** Firebase / Supabase / Ktor / Node.js (team decision)
- **Internet:** Required for streaming; offline mode supported for downloaded content and cached chat

2.5 Design and Implementation Constraints

- Architecture must be **Clean Architecture** with UI / Domain / Data layers
- UI state management must follow **UDF** via **StateFlow**
- No hardcoded colors, strings, or dimension values
- All large lists must use **Paging 3**
- Database and network calls must use **Dispatchers.IO**
- Real-time chat must use **WebSockets** — polling is prohibited
- Audio visualizer must be built with **Compose Canvas** — GIFs and Lottie are prohibited

2.6 Assumptions and Dependencies

- The backend will expose REST endpoints for CRUD operations and a WebSocket endpoint for chat
- At least 50 legally sourced songs with metadata will be seeded into the database before evaluation
- The ExoPlayer library (**media3**) will be used for all audio playback

- Hilt or Koin (one, chosen by team consensus) will be used throughout — mixing both is prohibited

Chapter 3

Functional Requirements

Each requirement is tagged with a priority level: **[MUST]** = mandatory for submission
[SHOULD] = expected for full marks **[COULD]** = bonus / creativity marks

3.1 Authentication & User Management

ID	Requirement	Priority
FR-01	Users shall be able to register with an email and password	[MUST]
FR-02	Users shall be able to log in with registered credentials	[MUST]
FR-03	Authentication state shall persist across app restarts	[MUST]
FR-04	Users shall be able to edit their display name and profile picture	[MUST]
FR-05	The system shall display a gold badge for premium users	[SHOULD]
FR-06	Users shall be able to initiate a simulated premium upgrade, setting <code>premium = true</code> in <code>DataStore</code>	[MUST]
FR-07	Users shall be able to log out from the Settings screen	[MUST]

3.2 Home Tab

ID	Requirement	Priority
FR-10	The home screen shall display a top carousel of new albums, daily recommendations, and trending songs	[MUST]
FR-11	Four quick-action buttons shall be present: Liked Songs, Recently Played, My Playlists, Top Artists	[MUST]
FR-12	Horizontal LazyRow sections shall display: Most Popular, New Releases, Global Playlists, Local Playlists	[MUST]
FR-13	All lists and cards shall display a shimmer skeleton animation while data loads	[MUST]

3.3 Search Tab

ID	Requirement	Priority
FR-20	The search input shall use <code>debounce()</code> to prevent network calls on every keystroke	[MUST]
FR-21	Search results shall be paginated using Paging 3	[MUST]
FR-22	Filter chips (Songs, Artists) shall refine search results	[SHOULD]
FR-23	Search history shall be stored in Room and displayed below the search bar	[MUST]
FR-24	Individual search history entries shall be deletable	[MUST]

3.4 Downloads Tab

ID	Requirement	Priority
FR-30	The downloads tab shall list all songs downloaded for offline use	[MUST]
FR-31	Downloaded songs shall support sorting (e.g. by title, artist, date)	[SHOULD]
FR-32	A downloaded song shall be deletable via swipe-to-dismiss	[MUST]

3.5 Playlists Tab

ID	Requirement	Priority
FR-40	Playlists shall be displayed in a two-column <code>LazyVerticalGrid</code>	[MUST]
FR-41	The tab shall group playlists into: International Music, Local Music, User Playlists	[MUST]
FR-42	Users shall be able to create, rename, and delete their own playlists	[MUST]
FR-43	Users shall be able to add and remove songs from their playlists	[MUST]
FR-44	Playlist song lists shall be paginated with Paging 3	[MUST]

3.6 Profile Tab

ID	Requirement	Priority
FR-50	The profile tab shall display the user's avatar with an edit button	[MUST]
FR-51	A premium badge shall be visible for premium-tier users	[SHOULD]
FR-52	An upgrade button shall trigger the simulated purchase flow	[MUST]
FR-53	The profile tab shall provide navigation to Language and Theme settings	[MUST]

3.7 Advanced Media Playback

ID	Requirement	Priority
FR-60	Audio playback shall be managed by a foreground service using <code>ExoPlayer</code> and <code>MediaSession</code>	[MUST]
FR-61	Music shall continue playing when the app is backgrounded or the screen is locked	[MUST]
FR-62	The system shall handle audio focus: pausing on incoming calls and resuming afterward	[MUST]
FR-63	Streamed songs shall be automatically cached using <code>CacheDataSource</code>	[MUST]
FR-64	Crossfade shall be applied when transitioning between songs	[SHOULD]
FR-65	A media notification with Play, Pause, and Next controls shall appear in the notification bar and lock screen	[MUST]
FR-66	A mini player composable shall float above the bottom navigation bar	[MUST]
FR-67	Playback speed shall be adjustable to 1×, 1.5×, and 2×	[SHOULD]
FR-68	A sleep timer shall allow playback to stop automatically after a user-specified duration, implemented with Coroutines	[SHOULD]

3.8 Now Playing Screen

ID	Requirement	Priority
FR-70	The transition from mini player to the full Now Playing screen shall use a shared element transition that enlarges the album cover	[MUST]
FR-71	The album artwork shall rotate continuously while playing and stop when paused	[MUST]
FR-72	The screen background color shall be dynamically derived from the album cover's dominant color using the Android Palette API	[MUST]
FR-73	An animated audio visualizer (equalizer or waveform) shall be rendered using Compose Canvas. GIFs and Lottie are explicitly prohibited	[MUST]
FR-74	All standard playback controls shall be accessible on the Now Playing screen	[MUST]

3.9 Business Logic — Premium & Downloads

ID	Requirement	Priority
FR-80	Regular users shall be able to stream all songs without restriction	[MUST]
FR-81	If a regular user attempts to download a song, an upgrade prompt shall be shown	[MUST]
FR-82	Premium users shall be able to download songs via WorkManager	[MUST]
FR-83	If a downloaded file exists locally, the player shall use it instead of streaming	[MUST]

3.10 Social Network & Real-Time Chat

ID	Requirement	Priority
FR-90	Users shall be able to search for other users	[MUST]
FR-91	Users shall be able to follow and unfollow other users	[MUST]
FR-92	Followed users' public playlists shall be visible	[MUST]
FR-93	Direct messages shall be transmitted in real time over WebSocket. Polling is prohibited	[MUST]
FR-94	Messages shall display status indicators: Sending (clock), Delivered (single check), Read (double check)	[MUST]
FR-95	A typing indicator shall be shown when the other user is composing a message	[SHOULD]
FR-96	Users shall be able to share a song in chat; the song shall appear as a custom mini-card that triggers playback on tap	[SHOULD]
FR-97	All chat messages shall be stored in Room for offline viewing	[MUST]
FR-98	Chat history shall be paginated using Paging 3	[MUST]

3.11 Local Data Management

ID	Requirement	Priority
FR-100	Room shall store: search history, liked songs, downloaded file paths, and offline chat messages	[MUST]
FR-101	DataStore shall persist: theme mode, font size, selected language, and premium status	[MUST]

Chapter 4

Non-Functional Requirements

4.1 Usability

ID	Requirement	Priority
NFR-01	The application shall support Persian (fa) and English (en) locales	[MUST]
NFR-02	Layout direction shall switch automatically — RTL for Persian, LTR for English	[MUST]
NFR-03	No text shall be hardcoded in source files; all strings shall reside in <code>strings.xml</code>	[MUST]
NFR-04	The app shall support Light and Dark themes, switchable via system settings and in-app controls	[MUST]
NFR-05	All shimmer loading animations shall appear before any list data is available	[MUST]
NFR-06	The UI shall exhibit fluid animations including scale-on-click and smooth screen transitions	[SHOULD]

4.2 Design System

ID	Requirement	Priority
NFR-10	A custom Material 3 theme shall define all Typography, Colors, and Shapes	[MUST]
NFR-11	No hardcoded color values, font sizes, or margin/padding values shall appear in the codebase	[MUST]
NFR-12	The application shall provide a custom adaptive launcher icon	[MUST]
NFR-13	A common <code>TopBar</code> composable shall appear on all main screens with: app logo and name (right), user avatar, notification icon, and settings icon (left)	[MUST]

4.3 Performance

ID	Requirement	Priority
NFR-20	No database or network operation shall execute on the Main thread; <code>Dispatchers.IO</code> shall be used	[MUST]
NFR-21	All large lists (search results, playlist songs, chat history) shall use Paging 3; loading an entire dataset into a single <code>LazyColumn</code> is not permitted	[MUST]
NFR-22	Songs streamed by users shall be automatically cached to reduce repeated network usage	[MUST]

4.4 Architecture & Maintainability

ID	Requirement	Priority
NFR-30	The project shall follow Clean Architecture with distinct UI, Domain, and Data layers	[MUST]
NFR-31	The UI pattern shall be MVVM or MVI, exposing screen state through <code>StateFlow<UiState></code>	[MUST]
NFR-32	One-shot events (e.g. Snackbar errors) shall be delivered via <code>SharedFlow</code> or <code>Channel</code>	[MUST]
NFR-33	Dependency Injection shall be implemented with Hilt or Koin (one framework, used consistently)	[MUST]
NFR-34	User interactions shall be modelled as <code>Intent</code> or <code>Event</code> objects	[SHOULD]

4.5 Reliability & Offline Support

ID	Requirement	Priority
NFR-40	The application shall handle audio focus transitions gracefully (pause, duck, resume)	[MUST]
NFR-41	The app shall display appropriate empty-state illustrations for: empty playlists, no search results, and no internet connection	[MUST]
NFR-42	Chat messages shall be accessible offline via Room even when the WebSocket connection is unavailable	[MUST]

Chapter 5

Use Cases

5.1 Actors

Actor	Description
Guest	Unauthenticated user; can only access the registration/login screen
Regular User	Authenticated, free-tier user
Premium User	Authenticated user with premium flag set to true
System	Background services: MediaSession, WorkManager, WebSocket client

5.2 Use Case Catalogue

5.2.1 Authentication Use Cases

ID	Name	Actor(s)	Brief Description
UC-01	Register	Guest	Create a new account with email and password
UC-02	Login	Guest	Authenticate with existing credentials
UC-03	Edit Profile	Regular User	Update display name and avatar
UC-04	Upgrade to Premium	Regular User	Simulate purchase, set premium = true
UC-05	Logout	Regular User	Clear session and return to login screen

5.2.2 Content & Playback Use Cases

ID	Name	Actor(s)	Brief Description
UC-10	Browse Home Feed	Regular User	View carousel, quick actions, and horizontal sections
UC-11	Search Songs/Artists	Regular User	Type query with debounce; view paginated results
UC-12	Play a Song	Regular User	Start streaming or play from local cache
UC-13	Control Playback	Regular User	Pause, skip, seek, adjust speed, set sleep timer
UC-14	Like / Unlike a Song	Regular User	Toggle like; persisted in Room
UC-15	Add Song to Playlist	Regular User	Select a playlist; song is associated
UC-16	View Now Playing	Regular User	Open full player from mini player

5.2.3 Download Use Cases

ID	Name	Actor(s)	Brief Description
UC-20	Download Song	Premium User	Enqueue download via WorkManager
UC-21	Play Offline	Premium User	Play from local file if available
UC-22	Delete Download	Premium User	Swipe-to-dismiss removes local file and Room entry
UC-23	Attempt Download (No Premium)	Regular User	System shows upgrade prompt

5.2.4 Social Use Cases

ID	Name	Actor(s)	Brief Description
UC-30	Search Users	Regular User	Find other users by name or handle
UC-31	Follow User	Regular User	Add user to following list
UC-32	Unfollow User	Regular User	Remove from following list
UC-33	View Public Playlists	Regular User	Browse followed user's public playlists
UC-40	Open DM Chat	Regular User	Open direct message thread with another user
UC-41	Send Message	Regular User	Send text message over WebSocket
UC-42	Receive Message	System	Deliver incoming WebSocket message to UI
UC-43	View Message Status	Regular User	Observe sent / delivered / read indicators
UC-44	See Typing Indicator	Regular User	View "...is typing" when peer is composing
UC-45	Share Song in Chat	Regular User	Send song as mini-card; tap to play
UC-46	View Offline Chat	Regular User	Read cached Room messages without connectivity

5.2.5 Settings Use Cases

ID	Name	Actor(s)	Brief Description
UC-50	Switch Language	Regular User	Toggle between English and Persian; layout direction updates automatically
UC-51	Switch Theme	Regular User	Choose Dark, Light, or System; persisted in DataStore
UC-52	Logout	Regular User	Invalidate session and navigate to login

5.3 Detailed Use Case: UC-12 — Play a Song

Use Case ID	UC-12
Name	Play a Song
Actor	Regular User / Premium User
Goal	Begin audio playback for a selected song
Preconditions	User is authenticated and on any screen showing a song item
Postconditions	Song is playing; mini player is visible; MediaSession is active
Main Success Scenario	
1	User taps a song item
2	System checks if a local downloaded file exists
3a	<i>[Local file exists]</i> System passes local URI to ExoPlayer
3b	<i>[No local file]</i> System fetches streaming URL from backend
4	ExoPlayer begins playback via the background MediaSession service
5	Mini player appears above bottom navigation bar
6	Media notification appears in notification bar and lock screen
Alternative Scenarios	
2a	No internet and no local file → system displays “No connection” empty state
4a	Audio focus lost (incoming call) → system pauses, resumes after call ends

5.4 Detailed Use Case: UC-41 — Send Message

Use Case ID	UC-41
Name	Send Message
Actor	Regular User, System (WebSocket)
Goal	Deliver a text message to another user in real time
Preconditions	User is in an open DM thread; WebSocket connection is established
Postconditions	Message is saved in Room (sender side) and delivered to recipient
Main Success Scenario	
1	User types a message; typing indicator is emitted to recipient
2	User taps Send
3	Message is persisted locally in Room with status <i>Sending</i> (clock icon)
4	Message is transmitted over WebSocket
5	Backend acknowledges; status updates to <i>Delivered</i> (single check)
6	Recipient opens and reads message; status updates to <i>Read</i> (double check)
Alternative Scenarios	
4a	WebSocket is disconnected → message is queued; retry on reconnection

Chapter 6

Architecture & Technical Design

6.1 Layered Architecture

The project shall implement Clean Architecture with three distinct layers:

UI Layer Composables, ViewModels, `UiState` sealed classes, Navigation graph, Design System (theme, typography, shapes).

Domain Layer Use Case classes, domain model data classes, repository interfaces, business rules (premium gate, smart playback routing).

Data Layer Repository implementations, Room DAOs and database, Retrofit/Ktor network services, DataStore, WorkManager workers, WebSocket manager, ExoPlayer service.

6.2 Technology Stack

Concern	Technology
UI	Jetpack Compose, Material 3
State management	MVVM or MVI, <code>StateFlow</code> , <code>SharedFlow</code> , <code>Channel</code>
Navigation	Jetpack Navigation Compose
DI	Hilt <i>or</i> Koin (team decision, one only)
Local DB	Room
Preferences	DataStore
Networking	Retrofit / Ktor client
Real-time	WebSocket (OkHttp or Ktor)
Media playback	ExoPlayer (media3), <code>MediaSession</code>
Background tasks	WorkManager
Paging	Paging 3
Image loading	Coil
Palette	Android Palette API
Canvas	Compose Canvas (visualizer)
Backend	Firebase / Supabase / Ktor / Node.js (team decision)

6.3 Key Integration Contracts

The following interfaces must be agreed upon by the team *before* any implementation begins, as they span module boundaries:

Contract	Between	What to define
SongRepository	Backend ↔ UI, Backend ↔ Media	Song data class shape, Flow return types
PlaybackService	Media ↔ UI	How composables bind to the service; mini player state exposure
UserRepository	Backend ↔ UI	Auth state Flow , follow status
ChatRepository	Backend ↔ UI	Message data class, WebSocket event model
DownloadRepository	Backend ↔ Media	Local file path format, WorkManager status Flow
UiState pattern	All	Shared sealed class structure for all screens

Critical Path Warning: Backend readiness blocks both the UI and Media teams. In Week 1, Collaborator A must provide stub/mock responses so the other two members can proceed in parallel.

Chapter 7

Work Breakdown Structure & Team Assignments

7.1 Team Overview

Member	Role	Primary Domain
Mahan Esmaeili	Team Lead	UI Layer, Social Features, State Management
Collaborator A	Backend Dev	Backend, Data Layer, DI, Database
Collaborator B	Media Dev	ExoPlayer Pipeline, Now Playing Screen, Notifications

7.2 Collaborator A — Backend & Data Layer

Owns: Backend services, all repository implementations, DI setup, database schema.

- Backend infrastructure setup (Firebase / Supabase / Ktor / Node.js)
- Database schema design and seeding with 50+ songs
- Retrofit / Ktor HTTP client and all API interface definitions
- Room database: schema for Songs, SearchHistory, LikedSongs, DownloadPaths, ChatMessages
- DataStore wrappers for theme, language, and premium status
- Hilt / Koin DI modules for all repositories, use cases, DB, and network
- Repository implementations: Auth, Songs, Playlists, Users
- WorkManager download worker with progress Flow
- WebSocket connection manager and ChatRepository

Week 1 Priority: Provide stub / mock implementations of all repositories so UI and Media teams can begin development without a live backend.

7.3 Collaborator B — Media Engine & Now Playing

Owns: All audio playback infrastructure, notifications, mini player, Now Playing screen.

- ExoPlayer + MediaSession foreground service setup

- Audio focus handling (call interruption, duck, resume)
- `CacheDataSource` smart caching integration
- Crossfade between tracks
- Playback speed control ($1\times$, $1.5\times$, $2\times$)
- Sleep timer via Coroutines
- Media notification with lock screen controls
- Mini Player composable (state from `PlaybackService`)
- Now Playing screen:
 - Rotating CD animation (`graphicsLayer` rotation)
 - Palette API dynamic background color
 - Canvas-based audio visualizer / equalizer bars
 - Shared element transition (mini $\rightarrow$ full player)

High-Risk Item: The Canvas audio visualizer is the most time-intensive UI component in the project. A bar equalizer with animated heights is sufficient. Prototype this in Week 2 and do not leave it to late stages.

7.4 Mahan Esmaeili — UI, Social & State Management

Owens: All main screens (except Now Playing), design system, localization, social features, real-time chat UI.

- Material 3 custom theme: Typography, Colors, Shapes
- `strings.xml` and full EN/FA localization with RTL/LTR auto-switching
- Dark / Light theme with in-app toggle (persisted via `DataStore`)
- Custom adaptive launcher icon
- Common `TopBar` composable
- Shimmer skeleton loading for all lists and cards
- Home Tab: carousel, quick actions, `LazyRow` sections
- Search Tab: debounce search, filter chips, Room-backed history
- Downloads Tab: sorted list, swipe-to-dismiss
- Playlists Tab: `LazyVerticalGrid` two-column layout
- Profile Tab: avatar edit, premium badge, upgrade flow, settings navigation
- Liked Songs / Recently Played screens
- Followed Artists & Users screen
- Settings screen: language, theme, logout
- Empty state illustrations/animations (no internet, empty playlist, no results)
- Social: user search UI, follow/unfollow
- Chat screen: WebSocket-driven DM thread, message status badges, typing indicator, song share mini-card

High-Risk Item: The Chat screen has hidden complexity (WebSocket event handling, three-state message status, offline Room fallback). Allocate at least one full week exclusively for the chat module.

Chapter 8

Development Timeline

Week	Milestone	Key Deliverables
1	Project Bootstrap	Architecture skeleton, DI modules, Room schema, backend stubs, design system tokens, shared data classes, team contracts defined
2	Core Infrastructure	Backend live & seeded; Auth screens; ExoPlayer basic playback; Home Tab skeleton with shimmer
3–4	Feature Development I	Search, Playlists, Downloads tabs; media notification; smart cache; Paging 3 integration across all lists
5–6	Feature Development II	Now Playing screen (all features); Profile & Settings; premium upgrade flow; WorkManager downloads
7	Social & Chat	Follow system; user search; WebSocket DM chat; message status; typing indicator; song sharing
8	Polish	Shared element transition; shimmer refinement; empty states; full localization pass; dark/light theme audit
9	QA & Hardening	Bug fixes; audio focus edge cases; offline mode testing; performance profiling; Paging 3 edge cases
10	Submission	Release APK build; demo video recording; final SRS review

8.1 Critical Path

The following sequence of work items has zero slack and must not be delayed:

1. **Week 1:** Team-wide contracts and DI module skeleton (blocks everything)
2. **Week 1–2:** Backend stubs → live backend (blocks streaming and chat)
3. **Week 2:** ExoPlayer service skeleton (blocks mini player and Now Playing)

4. **Week 7:** WebSocket chat (blocks social feature completion)
5. **Week 8:** Localization pass (must be done after all strings are added)

Appendix A

Expected Deliverables

Per the course specification, the following must be submitted:

1. **APK file** — signed release build of the final application
2. **Demo video** — short screen recording demonstrating all major features

Source code submission is not required by the course specification; however, the team should maintain a private version-controlled repository throughout development.

Appendix B

Song Metadata Schema

Each of the 50+ songs in the database must contain the following fields:

Field	Description
id	Unique identifier (UUID or integer)
title	Song title
artist_name	Performing artist or band name
cover_image_url	HTTPS URL to album art
audio_url	HTTPS URL to audio file (MP3 or AAC)

Acceptable sources: instrumental music, short clips, or other legally licensed content.

Appendix C

Three Team-Wide Decisions — Week 1 Checklist

The following decisions must be made unanimously in the first team meeting and documented. Changing them mid-project is extremely costly.

#	Decision	Options & Recommendation
1	Backend technology	Firebase (fastest setup), Supabase (PostgreSQL-based, easy REST + realtime), Ktor (most control, Kotlin-native), Node.js (most familiar for many).
2	DI framework	Hilt (Google-recommended for Android, more boilerplate) vs. Koin (simpler DSL). Choose one — mixing both is prohibited.
3	UI pattern	MVVM (simpler, familiar) vs. MVI (recommended for this project due to complex playback and chat state; steeper learning curve).